

High-Availability Reference Architecture

Early Access · MessageFoundry v0.3.2 · July 2026

MessageFoundry runs as a single process. High availability is added *around* that process — more copies of the same engine against the same store — rather than by changing it. This reference describes the **topology**: which failure domain each tier covers, who owns each layer, and what you have to supply yourself.

Read this alongside the Clustering document. This is the *shape* of an HA deployment. The **mechanism** — leader election, the self-fencing lease, the timing settings, the cluster status endpoints — belongs to the companion **Clustering** document and is not repeated here. Where the two appear to disagree, Clustering is authoritative.

The defining property of the design is that **the engine tier's coordination lives entirely in the shared SQL store**. There is no clustering broker, no quorum service, and no third arbiter to stand up: one row in the database decides who leads. (Choosing SQL Server Always On for the *database* tier does reintroduce Windows Server Failover Clustering and Active Directory — but that is the DBA's cluster, not the engine's.)

Recovery tiers

Three tiers ship. Each covers a different failure domain, and none of them is a scaling mechanism.

	Tier 1 — single node	Tier 2 — active/passive pair	Tier 3 — DR standby
Failure domain covered	process crash (the service restarts it)	loss of an engine host	loss of the pair, or of the whole site
Engines	one	two or more identical engines, one leader	one right-sized engine at the recovery site
Store	SQLite, PostgreSQL or SQL Server	PostgreSQL or SQL Server, shared by every node	seeded at the recovery site from a backup
Capacity after the failure	none until restarted	full — the standby runs the same graph	deliberately degraded — only feeds at or above a priority tier start
Activation	operator restart	automatic: a standby acquires leadership	manual and RBAC-gated
Operator load balancer	optional	required	required
Status	the default	opt-in, built	opt-in, built

Tier 2 is what most adopters mean by "HA", and it is the supported HA topology. A horizontal **active/active** model — the graph running concurrently on every node — is **not offered**; it was dropped and its code removed.

Tiers 2 and 3 are alternatives, not a stack you enable together: a node configured to come up under the DR run profile *and* as a cluster member is **refused at configuration load**. A warm engine at the recovery site should instead join the cluster as a **non-promotable** member (below).

What HA is not: engine shards

An **engine shard** is one of N `serve --shard` processes that split the **inbound connections** between them over **one unified store**. It buys CPU parallelism, not redundancy — each engine shard owns a disjoint slice of intake, so losing one takes its feeds down with it. It is also **mutually exclusive with clustering**: combining `--shard` with an enabled cluster is refused at startup, because the leadership lease is store-wide and would transfer across engine-shard ids. Above one engine shard a server database is required so the store stays unified, and the multi-engine-shard topology is **not yet certified as a production topology**. Never read an engine-shard count as an availability figure. (A *database* shard — splitting the store itself — is a different thing again, and is declined by design.)

Requirements (Tier 2)

A clustered deployment requires, and enforces at configuration load:

- A **server-class shared store** — PostgreSQL or SQL Server. SQLite is a single-file, single-node store and is refused: the cluster needs the shared membership and lease rows a server database provides. Both backends run the same leadership lease and are at parity.
- **Every node points at the same database** (same server, database and schema) and runs the **same** configuration directory.
- A **connection pool with headroom** — a clustered node drives background work (membership and lease renewal, the leader's reclaim sweep, the per-stage workers) alongside request traffic. A pool of one is refused; three or more is the recommendation.

All three are caught at configuration load, not discovered at failover. The same `serve` command starts on each host; nodes self-identify, and you only pin a node identity when you want it stable across restarts.

The engine tier — one arbiter, no quorum

Leadership is an **application-level lease held as a row in the shared database**. Exactly one node holds it and runs the whole message graph; every other node is a **warm standby** that binds no listener ports and runs no workers, keeps its membership heartbeat alive, and brings the full graph up the moment it acquires leadership.

Two architectural consequences are worth stating plainly, because they are what make this topology cheap to stand up:

- **There is no quorum and no witness at the engine tier.** The database row is the arbiter, so a two-node cluster is a complete cluster — no odd-vote arithmetic, no third tie-breaker host. Quorum re-enters only if your *database* tier is itself a Windows failover cluster; that is the DBA's design, covered below.

- **There is no node-to-node socket.** Coordination rides the store connection each node already holds, so there is no cluster network to firewall, tune, or encrypt separately.

The safety argument — lease expiry evaluated on the database's own clock, a leader that self-fences before its lease can expire, and a durable epoch check inside the claim transaction — is set out in the Clustering document. Take it as read here: **exactly one node processes at a time**, and per-lane FIFO ordering survives a failover.

Keeping leadership at the preferred site

By default any promotable node may take over an expired lease, first to claim it wins. In a two-site deployment that is usually not what you want: a node at the recovery site winning a routine take-over drags every commit across the WAN, and leadership does **not** fail back on its own. Two per-node settings shape the outcome:

- **A take-over handicap.** A node can be told to wait a configured number of seconds *past* the lease-expiry instant before it may claim an expired lease. Preferred-site nodes keep the handicap at zero and win the routine race; a remote node takes over only if no preferred node claims. It never delays the sitting leader's renewal and only ever makes a node claim *later*, so it cannot open a two-leader window.
- **A non-promotable standby.** A node can be marked as never eligible for leadership: it never inserts, takes over, or renews the lease, and steps down cleanly if it somehow holds it. That is the right shape for a warm engine at the recovery site. At least one promotable node must exist, or nothing ever leads and the graph never drains.

Neither setting substitutes for a decision about *where* leadership should live. Make that decision explicitly, and write the fallback step into your runbook.

The client-facing address (operator-supplied)

Clients reach "the engine" through a **floating VIP or Layer-4 load balancer that you supply**, not a fixed hostname. MessageFoundry does not ship this component and does not move an IP address itself.

The mechanism is deliberately simple, and it falls out of leader-gating:

- **One VIP per inbound port, health-checked with a plain TCP connect** to that listener port.
- **Only the leader binds the port**, so the check passes only on the leader and the VIP lands there automatically.
- **On failover the new leader binds and the old socket closes**, so the VIP follows with no manual intervention.
- **Configure partners to reconnect on drop** — standard MLLP client behaviour. On failover they see a connection drop and reconnect through the VIP.

The engine API — the console and IDE control/read plane — is up on **every** node, so an API-facing VIP can health-check the unauthenticated liveness endpoint. That endpoint answers on every node and therefore cannot select the leader; to pin an operation to the leader, read the cluster's read-only status endpoints instead. Those require a monitoring permission, so a load balancer probing them needs a least-privilege token injected — an unauthenticated probe is rejected everywhere and black-holes traffic. The web console surfaces the current role, leader and lease holder from the same endpoints.

Not built. An *engine-managed* VIP, where the engine binds and releases the address itself in lockstep with the lease, is a recorded design (Windows-only at v1) with **no code today**. Architect for the external VIP or load balancer: it is the cross-platform path, and it remains the recommendation for the strictest split-brain guarantee even after an engine-managed option ships.

The database tier is delegated to the database

MessageFoundry does **not** replicate the store, and on a server database it does not back it up either. **Database-tier availability is the database's job**, using its own mechanism:

- **PostgreSQL** — streaming replication with your chosen failover tooling.
- **SQL Server** — Always On availability groups.

Point every engine node at the database's HA endpoint (the listener or failover address). Engine failover and database failover are then **separate axes**: the engine rides through a database failover as a bounded store outage rather than a leadership change, provided that outage stays comfortably shorter than the margin between the heartbeat interval and the self-fence timeout. Nothing acknowledged is lost either way — a failed commit means the sender never got an acknowledgement and still holds the message.

Four points belong in the design rather than the runbook:

- **Synchronous commit belongs inside one site.** The staged pipeline makes several durable commits per message, and within an ordered lane they run serially — so every millisecond added to the commit round trip is paid several times over, per message. A synchronous replica across a WAN link multiplies the link's round trip into each of those commits. Keep synchronous replicas local, keep the remote replica asynchronous, and accept a small, *measured* recovery-point objective there instead.
- **If the database tier is a Windows failover cluster, quorum comes back — theirs, not ours.** An availability-group design wants an odd vote count with a witness outside both data centres, and zero votes at the recovery site so a flaky link can never cost the primary site its quorum. The engine has no dependency on any of it.
- **Cross-subnet listeners need DNS-side care.** The store connection takes one Always On-aware ODBC keyword, and it has shipped: `[store].multi_subnet_failover` emits `MultiSubnetFailover=Yes`, so the driver races the subnets instead of serially waiting each one out. It is **opt-in and defaults to off**, it has not yet been exercised against a live cross-subnet availability group, and it covers the *store* connection only — the separate database-lookup connector builds its own connection string and does not emit it. So set the keyword *and* configure the listener to register only the active subnet's address with a short record TTL, then drill the cross-site connect path.
- **Backups and restores are yours.** On a server database the engine's DR backup covers **configuration only**; restoring the database itself is delegated to the DBA. Promotion at the recovery site is fail-closed on that: it requires an explicit per-activation attestation that the database was restored, plus a live provenance check on the restored store, and refuses otherwise.

Third tier — the DR standby

Beyond the HA pair there is an opt-in **DR standby**: a right-sized engine at the recovery site that comes up under a **restricted run profile**. It is the inverse of a scale-out — it deliberately runs *less*:

- it starts **only** the connections at or above a configured priority tier; everything below reports as filtered rather than starting;
- it **seeds its store from an encrypted backup**, and **fails closed** if the store's key material is not available at the recovery site;
- **activation is manual** — an RBAC-gated, audited operator action; automatic activation is rejected at configuration load;
- it can be fenced by an "acquire the address or abort" hook, so it never starts binding feeds behind a load balancer that has not actually moved.

Use it when losing the whole primary site is in scope and running a full-capacity second site is not. Do not also run it as a cluster member — that combination is refused at configuration load.

What a failover actually costs

Failover is **not instantaneous**. Plan for a promotion window rather than zero downtime.

- **Clean stop** (planned switchover or graceful shutdown) — the leaving leader releases its lease, so a standby is promoted on its next heartbeat.
- **Crash or network partition** — the dead leader's lease has to age out, and a standby acquires once it expires. A partitioned old leader stops processing before that point, so it never overlaps its successor.

Across the window, nothing acknowledged is lost. The engine acknowledges a message only after it is durably committed, so a sender is never told "accepted" for a message that was dropped. The promoted leader recovers its predecessor's stranded in-flight work, and the stranded head of each lane is reclaimed before anything newer on that lane, so ordering survives. What you owe in return: **delivery is at-least-once, so downstream endpoints must be idempotent** — a message interrupted mid-delivery is delivered again.

Quantify *your* window from the published failover benchmark rather than assuming one; the lease timings that govern it, and the rule for tuning them to your network, are in the Clustering document.

Operational guidance

- **Keep node clocks synced (NTP)**. Leadership is decided on the database's clock, so skew cannot change who may lead — but the per-row recovery leases use node wall-clock, so keep the hosts sane.
- **Deploy identical configuration to every node**. Each node loads its graph from its own configuration directory; the cluster coordinates the reload *version*, not the files. A divergent node is not cosmetic — on promotion it can dead-letter messages for destinations it does not know about.
- **Apply configuration changes as a coordinated restart, not a rolling one**, so no two nodes ever run divergent graphs across the change window.

- **Drill the failover you depend on** — a planned database failover, an engine promotion, and, if you run one, the recovery-site activation including the address move and any sender cutover.

Transport security still applies

HA does not change the transport posture. MessageFoundry binds loopback by default and **refuses at startup** a non-loopback API bind without TLS or a declared trusted terminator; a non-loopback MLLP, HTTP or DICOM listener without TLS is refused the same way. When you expose a listener off loopback — which any real data-plane feed requires — enable TLS on that channel: the API supports in-process TLS (a TLS 1.2 floor, with optional client-certificate mTLS) or an upstream terminator, and MLLP supports per-connection TLS and optional mutual TLS. Raw TCP and X12 listeners have **no TLS of their own**, so a non-loopback bind of one is refused outright — keep them bound to loopback behind a TLS-terminating proxy, or segment them at the OS/firewall level. Inter-node coordination rides the shared store connection and adds **no node-to-node socket**, so securing the store connection secures the cluster traffic with it. This is deployment guidance for the environment, not a limitation of the HA model.

Related

- **Clustering** (the companion document in this set) — the HA mechanism: leader lease, graph gating, lease-timing tuning, and the cluster status endpoints. [CLUSTERING.md](#)
- [AOAG-DEPLOYMENT.md](#) — a worked two-site reference deployment on SQL Server Always On, including placement, quorum and the failure/runbook matrix.
- [DEPLOYMENT.md](#) — network exposure and the per-channel TLS posture matrix.
- [SYSTEM-REQUIREMENTS.md](#) — sizing, measured throughput tiers, and which topologies are supported.
- [CONFIGURATION.md](#) — the full store, cluster and DR settings catalog.